

EXPERIMENT 5:

Solve 8-Queens Problem with suitable assumptions

EVALUATION

Writeup (3M)	
Execution (5M)	
Viva Voce (2M)	
Total Marks (10M)	
Faculty Signature and Date	

5.1. AIM

To study and implement the **8-Queens Problem** using appropriate Artificial Intelligence search techniques, and to demonstrate how intelligent search strategies — such as **Backtracking**, **Constraint Satisfaction**, and **Local Search** — can efficiently explore a large problem space to find valid, conflict-free placements of 8 queens on an 8×8 chessboard such that no two queens attack each other.

Specific Objectives

1. To **formulate** the 8-Queens Problem as a state-space search problem by defining states, operators, initial state, and goal state.
2. To **identify and apply** constraints — row, column, and diagonal — to prune the search space and eliminate invalid configurations early.
3. To **implement** a backtracking algorithm that systematically places queens column by column and backtracks upon detecting a conflict.
4. To **compare** the efficiency of uninformed brute-force search against constraint-based backtracking in terms of the number of states explored.
5. To **observe and analyze** the behavior of local search methods such as Hill Climbing on the 8-Queens Problem, including the occurrence of local optima and plateaus.
6. To **verify** the correctness of the solution by confirming that all row, column, and diagonal constraints are satisfied in the final board configuration.
7. To **generalize** the solution approach to the N-Queens Problem and study how computational complexity scales with increasing N.

5.2. Description

The 8-Queens Problem is a classical combinatorial puzzle in Artificial Intelligence that involves placing 8 queens on an 8×8 chessboard such that no two queens threaten each other. In the game of chess, a queen can attack any other piece that lies in the same row, same column, or along either diagonal. Therefore, the objective is to find a configuration where all 8 queens coexist peacefully — no queen attacks any other.

This problem serves as an excellent model for studying **search strategies**, **constraint satisfaction**, and **optimization techniques** in AI. Despite its simple rules, the problem has a large search space of $8^8 = 16,777,216$ possible arrangements, of which only **92 distinct solutions** exist. This makes it non-trivial enough to justify intelligent search methods, yet small enough to validate results by hand or by brute force.

The problem generalizes to the **N-Queens Problem**, where N queens are placed on an N×N board, making it scalable for benchmarking different AI algorithms.

5.3. Assumptions and Constraints

Assumptions

- The board is a standard **8×8 chessboard** with rows and columns numbered 1 through 8.
- All **8 queens are identical** and indistinguishable from one another.
- Exactly **one queen is placed per column**, reducing the search space from 8^8 to $8! = 40,320$. This is a standard simplifying assumption since any valid solution must have exactly one queen in each column.
- The problem seeks **any one valid solution** unless stated otherwise. Finding all 92 solutions is treated as an extension.
- The board is treated as a **static environment** — no pieces other than queens are present.
- The algorithm operates with **complete and observable** knowledge of the board state at every step.

Constraints

- **Row Constraint** — No two queens may occupy the same row.
- **Column Constraint** — No two queens may occupy the same column. (Enforced implicitly by the one-queen-per-column assumption.)
- **Diagonal Constraint (Left to Right)** — No two queens may lie on the same left diagonal. Formally, for queens at positions (r_1, c_1) and (r_2, c_2) :

$$r_1 - c_1 \neq r_2 - c_2$$

- **Diagonal Constraint (Right to Left)** — No two queens may lie on the same right diagonal. Formally:

$$r_1 + c_1 \neq r_2 + c_2$$

- **Placement Constraint** — Every column must contain exactly one queen; no column may be left empty or hold more than one queen.
- **Board Boundary Constraint** — All queens must be placed within the 8×8 grid; no placement outside the board is valid.

5.4. BRUTE FORCE TECHNIQUE

5.4.1. Algorithm: Brute_Force_8Queens

Pre-condition

- A chessboard of size 8×8 is taken
- 8 queens are to be placed on the board
- Each queen must be placed in a separate column
- No two queens should attack each other

What is a Permutation here?

Since we place exactly one queen per column, we only need to decide **which row** each queen goes in. A permutation of $[0, 1, 2, 3, 4, 5, 6, 7]$ gives us one possible arrangement where each row is used exactly once. All such permutations are generated and tested one by one.

Steps

Step 1 : Start

Step 2 : Generate all possible arrangements of row positions for 8 queens — one queen per column. Since each row can only be used once, this gives $8! = 40,320$ arrangements in total.

Step 3 : Pick the first arrangement that has not been tested yet.

Step 4 : Check every possible pair of queens in the current arrangement. For each pair, perform the following two checks:

- **Row Check** — If both queens are in the same row, this arrangement is invalid.
- **Diagonal Check** — If the difference in their row positions equals the difference in their column positions, they are on the same diagonal and this arrangement is invalid.

Step 5 : If any pair of queens fails either check in Step 4, mark this arrangement as **invalid** and move to Step 7.

Step 6 : If all pairs of queens pass both checks successfully, mark this arrangement as a **valid solution**. Print the board configuration and increase the solution count by 1.

Step 7 : Check if all 40,320 arrangements have been tested.

- If **No** — go back to Step 3 and pick the next untested arrangement.
- If **Yes** — proceed to Step 8.

Step 8 : Print the total number of valid solutions found.

Step 9 : Stop

Why $8!$ and not 8^8 ?

If we allowed any queen to go in any row without restriction, we would have $8^8 = 16,777,216$ arrangements to test. But since we know that in any valid solution each row must contain exactly one queen, we only generate **permutations** of rows — reducing the search space to $8! = 40,320$. This is the only optimization brute force makes.

Key Characteristics

Property	Brute Force
Total arrangements tested	40,320
Conflict detection	Only after full placement
Invalid branches pruned	None — all are tested
Solutions found	All 92
Intelligence used	None — pure exhaustive search

5.4.2. Program

<https://github.com/mayurjambit/BAD402>

5.5. BACK TRACKING

5.5.1 Algorithm: Backtracking_8Queens

Algorithm — Backtracking Approach

Pre-condition

- A chessboard of size 8×8 is taken
- 8 queens are to be placed on the board
- Queens are placed **one column at a time** from left to right
- The board is initially empty — no queens are placed

What is Backtracking?

Backtracking is a technique where we build the solution **step by step**. At each step we check whether the current placement is safe. If it is safe we move forward to the next column. If it is not safe or if no safe row exists in the current column, we **go back** to the previous column and try the next available row there. This process of going back is called **backtracking**.

What is a Safe Position?

A position (row, column) is called safe if the queen placed there is not attacked by any previously placed queen. This means:

- No previously placed queen is in the **same row**
- No previously placed queen is on the **same diagonal**

Column conflict does not need to be checked because we place exactly one queen per column by design.

Steps

Step 1 : Start

Step 2 : Set the current column to **Column 0** (the leftmost column). The board is empty at this point.

Step 3 : If all 8 queens have been successfully placed (i.e. current column = 8), then a valid solution has been found. Print the board configuration and record it as a solution. Then go back to the previous column and continue trying remaining rows there — this finds all solutions.

Step 4 : In the current column, start trying rows from **Row 0 to Row 7** one by one.

Step 5 : For the current row being tried, check whether placing a queen at this position is **safe** by examining all previously placed queens:

- **Row Check** — If any previously placed queen occupies the same row, this position is not safe.

- **Diagonal Check** — If the absolute difference between the row numbers equals the absolute difference between the column numbers for any previously placed queen, this position is on a diagonal and is not safe.

Step 6 : If the position is **not safe**, skip this row and try the next row in the same column. Go back to Step 5.

Step 7 : If the position is **safe**, place the queen at this position. Record the row number for this column.

Step 8 : Move to the **next column** (current column + 1) and go back to Step 3 to repeat the process for the next column.

Step 9 : If all rows from 0 to 7 have been tried in the current column and none of them were safe, it means the current partial arrangement cannot lead to a valid solution. **Remove the queen** from the current column and go back to the **previous column**.

Step 10 : In the previous column, continue trying rows that have not yet been tried (rows after the one that was last placed). Go back to Step 5.

Step 11 : If backtracking reaches Column 0 and all rows have been exhausted there as well, all possible solutions have been found. Print the total number of solutions found.

Step 12 : Stop

Backtracking in Action — Trace Example

Column 0 → Try Row 0 → Safe → Place queen → Move to Column 1

Column 1 → Try Row 0 → Not safe (same diagonal) → Skip

Try Row 1 → Not safe (same diagonal) → Skip

Try Row 2 → Safe → Place queen → Move to Column 2

Column 2 → Try Row 0 → Not safe → Skip

Try Row 1 → Not safe → Skip

...

Try Row 7 → Not safe → No rows left → BACKTRACK

Column 1 → Remove queen from Row 2 → Try Row 3 → Safe → Place queen

Move to Column 2 → Continue...

Key Difference from Brute Force

Situation	Brute Force	Backtracking
Conflict detected at Column 3	Continues building all 8 columns then checks	Immediately abandons and tries next row in Column 3
Invalid partial arrangement	Still generates all arrangements built on it	Prunes the entire subtree — never generates them
Total states explored	40,320	~15,720

The fundamental advantage of backtracking is that the moment a conflict is detected in any column, **all arrangements that would have been built on top of that conflict are discarded without being generated**. This is called **pruning** and it is what makes backtracking significantly more efficient than brute force.

5.5.2. Program

<https://github.com/mayurjambit/BAD402>

5.6. HILL CLIMBING

Hill Climbing is a **local search algorithm** that belongs to the family of heuristic search techniques in Artificial Intelligence. Unlike backtracking which systematically explores the search tree, hill climbing works on a **complete state formulation** — it starts with an initial configuration of all 8 queens already placed on the board and then iteratively improves it.

The algorithm gets its name from the analogy of climbing a hill — at every step, you look at your neighbouring positions and move to the one that is **higher** (better) than your current position. You stop when you reach a position where no neighbour is better — a **peak**. In minimization problems like 8-Queens, we descend instead, making it effectively **Hill Descending**, but the term Hill Climbing is used universally.

5.6.1. Problem Specification

How it Works for 8-Queens

- **State** — A complete board configuration with exactly one queen in each column, placed at some row.
- **Heuristic $h(n)$** — The number of pairs of queens that are attacking each other. A perfect solution has $h(n) = 0$.
- **Neighbour** — Any state reachable by moving a single queen to a different row within its column. For an 8×8 board, each state has $8 \times 7 = 56$ neighbours.
- **Move** — Always move to the neighbour with the **lowest $h(n)$** (fewest conflicts).
- **Termination** — Stop when $h(n) = 0$ (solution found) or no neighbour has a lower $h(n)$ than the current state (local minimum reached).

5.6.2. ALGORITHM

Algorithm — Hill Climbing (No Random Restart)

Pre-condition

- A chessboard of size 8×8 is taken
- One queen is placed in each column at a random row
- This gives us a starting board with 8 queens already placed

What is $h(n)$?

$h(n)$ is the **number of pairs of queens attacking each other** on the current board. Our goal is to reduce $h(n)$ to **zero**.

Steps

Step 1 : Start

Step 2 : Place one queen in each column at a fixed starting row position. This is our **initial board state**.

Step 3 : Count the number of attacking queen pairs on the current board. This value is called $h(n)$. Print the board and its $h(n)$.

Step 4 : If $h(n) = 0$, then no queens are attacking each other. The board is a **valid solution**. Print the solution and **Stop**.

Step 5 : For each queen on the board, try moving it to every other row within its own column — one move at a time. This gives **56 possible neighbouring boards** (7 moves $\times$ 8 columns).

Step 6 : Calculate $h(n)$ for each of the 56 neighbouring boards.

Step 7 : Find the neighbour that has the **lowest $h(n)$** among all 56 neighbours. Call this the **best neighbour**.

Step 8 : Compare the best neighbour's $h(n)$ with the current board's $h(n)$.

Step 9 : If the best neighbour's $h(n)$ is **less than** the current $h(n)$, then move to that neighbour. That board now becomes the **current board**. Go back to **Step 3**.

Step 10 : If the best neighbour's $h(n)$ is **equal to or greater than** the current $h(n)$, then no move improves the situation. The algorithm is **stuck at a local minimum**. Print the current board and its $h(n)$.

Step 11 : Print that the algorithm has failed to find a solution from this starting position because it is trapped at a local minimum where no single queen move leads to improvement.

Step 12 : Stop.

Why does it stop at Step 10?

Because hill climbing has **no memory** and **no way to go backwards**. It can only move to a better state. When no better state exists nearby, it simply gives up. This is the fundamental limitation of pure hill climbing.

5.6.3. Program

<https://github.com/mayurjambit/BAD402>

5.7. GENETIC ALGORITHM

What is a Genetic Algorithm?

A Genetic Algorithm (GA) is a search technique inspired by the process of **natural evolution**. It mimics how living organisms evolve over generations to become better adapted to their environment. In AI, it is used to find good solutions to optimization problems by evolving a **population of candidate solutions** over many generations.

Key Concepts Before the Algorithm

Term	Meaning in GA	Meaning in 8-Queens
Individual	One candidate solution	One board configuration
Population	A group of individuals	A set of board configurations
Chromosome	Encoded form of a solution	A list of 8 row positions e.g. [2,4,6,1,3,5,7,0]
Gene	One element of a chromosome	Row position of one queen
Fitness	How good a solution is	Number of non-attacking pairs (higher = better)
Selection	Choosing parents to reproduce	Pick boards with higher fitness
Crossover	Combining two parents	Merge two board configurations
Mutation	Small random change	Move one queen to a random row
Generation	One cycle of evolution	One round of selection, crossover, mutation

Fitness Function

The maximum number of non-attacking queen pairs on an 8-queen board is **28** (since there are $8 \times 7 / 2 = 28$ pairs total).

$\text{Fitness}(\text{board}) = 28 - \text{number of attacking pairs}$

A perfect solution has **Fitness = 28**.

5.7.1. Algorithm — Simple Steps

Step 1 : Start

Step 2 : Create an initial population of N random boards. Each board is a list of 8 numbers representing the row of the queen in each column.

Step 3 : For each board in the population, calculate its **fitness score** using the formula:
 $\text{Fitness} = 28 - \text{number of attacking pairs}$.

Step 4 : If any board has fitness = 28, it is a valid solution with zero conflicts. Print the board and Stop.

Step 5 : Selection — Choose two parent boards from the population. Boards with higher fitness scores have a higher chance of being selected. This is called fitness-proportionate selection.

Step 6 : Crossover — Pick a random crossover point between position 1 and 7. Create a child board by taking genes from Parent 1 up to the crossover point, and genes from Parent 2 after the crossover point.

Step 7 : Mutation — With a small probability (e.g. 10%), randomly change one gene in the child board to a new random row value.

Step 8 : Repeat Steps 5, 6, and 7 until a new population of N children is created.

Step 9 : Replace the old population with the new population of children.

Step 10 : Increment the generation counter. If the maximum number of generations has been reached without a solution, print failure and Stop. Otherwise go back to Step 3.

Step 11 : Stop

5.7.2. Program

<https://github.com/mayurjambit/BAD402>

5.8. Output

5.8.1. BRUTE FORCE (Enter Number of solutions to display = 1, then write the output)

5.8.2. Back Tracking (Print only the Total nodes explored and total valid solutions)

5.8.3. Hill Climbing (Write only the Summary portion of the output)

5.8.4. Output Genetic Algorithm (Write initial specification of the GA, then write chromosomes, fitness etc seen at the end)